Computational Data Analysis (02582)

Decomposition methods for unsupervised learning

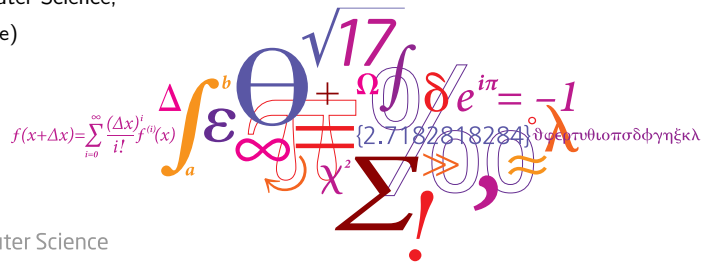
Sneha Das

Assistant Professor

Department of Applied Mathematics and Computer Science,

Technical University of Denmark (DTU Compute)

(*Slides adapted: Morten Mørup)*



DTU Compute

Department of Applied Mathematics and Computer Science

Outline I

- Non-negative Matrix Factorization
- Independent Component analysis
- Archetypal Analysis
- Sparse coding

Factor Analysis and Matrix Decomposition

The Core Concept:

Factor analysis and matrix decomposition methods aim to approximate a complex data matrix $\mathbf{X}$ as the product of lower-dimensional factors: $\mathbf{X} \approx \mathbf{WH}$. This reveals hidden structures, latent variables, or meaningful representations of the original data.

Factor Analysis and Matrix Decomposition

The Core Concept:

Factor analysis and matrix decomposition methods aim to approximate a complex data matrix $\mathbf{X}$ as the product of lower-dimensional factors: $\mathbf{X} \approx \mathbf{WH}$. This reveals hidden structures, latent variables, or meaningful representations of the original data.

Four Methods, Four Unique Constraints:

- **Non-negative Matrix Factorization (NMF):** Constrains all learned elements to be strictly positive (≥ 0), forcing an additive, *parts-based* representation.
- **Independent Component Analysis (ICA):** Assumes underlying sources are *statistically independent* and non-Gaussian, rotating data to unmix overlapping signals.
- **Archetypal Analysis (AA):** Restricts components to be convex mixtures of the data itself, identifying the *extreme prototypes* (the convex hull) rather than averages.
- **Sparse Coding (SC):** Learns an overcomplete dictionary where each observation must be reconstructed using a *strictly sparse* combination (mostly zeros) of basis vectors.

Think - Pair - Share: The Mechanics of Factorization

Vevox Input Required

Suppose we have a simple 2×2 data matrix $\mathbf{X}$ that we want to perfectly decompose into a rank-1 basis matrix $\mathbf{W}$ and an activation matrix $\mathbf{H}$:

$$\mathbf{X} = \mathbf{WH} \implies \begin{bmatrix} 4 & 6 \\ 6 & 9 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \begin{bmatrix} h_1 & h_2 \end{bmatrix}$$

Task

What are the values of h_1 and h_2 required to reconstruct $\mathbf{X}$?

- **Calculate h_1 and h_2 in pairs.**
- **Share:** Submit your answer pair as (h_1, h_2) in the Vevox poll!

Solution: The Mechanics of Factorization

The Matrix Multiplication:

$$\begin{bmatrix} 2 \\ 3 \end{bmatrix} \begin{bmatrix} h_1 & h_2 \end{bmatrix} = \begin{bmatrix} 2 \cdot h_1 & 2 \cdot h_2 \\ 3 \cdot h_1 & 3 \cdot h_2 \end{bmatrix}$$

Solution: The Mechanics of Factorization

The Matrix Multiplication:

$$\begin{bmatrix} 2 \\ 3 \end{bmatrix} \begin{bmatrix} h_1 & h_2 \end{bmatrix} = \begin{bmatrix} 2 \cdot h_1 & 2 \cdot h_2 \\ 3 \cdot h_1 & 3 \cdot h_2 \end{bmatrix}$$

Matching it to the Data Matrix $\mathbf{X}$:

$$\begin{bmatrix} 2 \cdot h_1 & 2 \cdot h_2 \\ 3 \cdot h_1 & 3 \cdot h_2 \end{bmatrix} = \begin{bmatrix} 4 & 6 \\ 6 & 9 \end{bmatrix}$$

Solution: The Mechanics of Factorization

The Matrix Multiplication:

$$\begin{bmatrix} 2 \\ 3 \end{bmatrix} \begin{bmatrix} h_1 & h_2 \end{bmatrix} = \begin{bmatrix} 2 \cdot h_1 & 2 \cdot h_2 \\ 3 \cdot h_1 & 3 \cdot h_2 \end{bmatrix}$$

Matching it to the Data Matrix $\mathbf{X}$:

$$\begin{bmatrix} 2 \cdot h_1 & 2 \cdot h_2 \\ 3 \cdot h_1 & 3 \cdot h_2 \end{bmatrix} = \begin{bmatrix} 4 & 6 \\ 6 & 9 \end{bmatrix}$$

The Solution

By solving the simple linear equations (e.g., $2 \cdot h_1 = 4$ and $2 \cdot h_2 = 6$), we find:

$$h_1 = 2, \quad h_2 = 3$$

Result: The activation matrix is $\mathbf{H} = \begin{bmatrix} 2 & 3 \end{bmatrix}$.

Takeaway: *The basis $\mathbf{W}$, and the activations $\mathbf{H}$ provide the scaling 'intensity' of that pattern for each column in $\mathbf{X}$.*

- Non-negative Matrix Factorization
- Independent Component analysis
- Archetypal Analysis
- Sparse coding

Non-negative matrix factorization (NMF)

Given a non-negative data matrix $\mathbf{X} \in \mathbb{R}_+^{I \times J}$, we seek a decomposition into two lower-rank, non-negative matrices:

$$\mathbf{X} \approx \mathbf{WH}$$

- $\mathbf{W} \in \mathbb{R}_+^{I \times K}$: The **Basis** or Dictionary (latent features).
- $\mathbf{H} \in \mathbb{R}_+^{K \times J}$: The **Coefficients** or Activations (how much of each latent feature contributes to yield X).
- $K \ll \min(I, J)$: The number of components.

The Constraint: Every element $w_{ik} \geq 0$ and $h_{kj} \geq 0$.

Why Non-Negativity?

Why Non-Negativity?

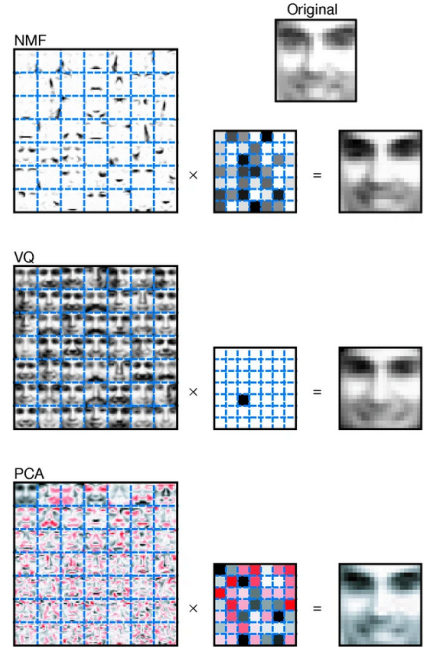
1. Physical realism: In many fields, negative values are not realistic. Examples:
   - **Audio:** Power spectra/energy.
   - **Imaging:** Pixel intensities.
   - **Text:** Word counts in a document.
2. Parts-Based Representation: Because we cannot subtract features (only add them), the model is forced to learn 'parts' (e.g: a specific note in music or a nose on a face) rather than 'holistic' ghostly averages like in PCA.

The Seminal Papers of NMF

The 'Parts' Concept

Learning the parts of objects by non-negative matrix factorization
Lee & Seung, Nature (1999)

- **Core Contribution:** Introduced the intuitive concept of 'parts-based representation'.
- **Demonstration:** Showed NMF learning localized facial features (eyes, noses) vs. holistic 'eigenfaces' in PCA.
- **Impact:** Popularized the algorithm across disciplines by highlighting its psychological and biological plausibility.



The Seminal Papers of NMF

The Algorithms

Algorithms for Non-negative Matrix Factorization
Lee & Seung, NIPS (2000)

- **Core Contribution:** Derived the explicit *Multiplicative Update Rules* for both Frobenius norm and KL Divergence.
- **Mathematical Rigor:** Provided formal proofs of convergence using auxiliary functions (akin to EM algorithms).
- **Impact:** Established the standard, easy-to-implement computational foundation used for decades.

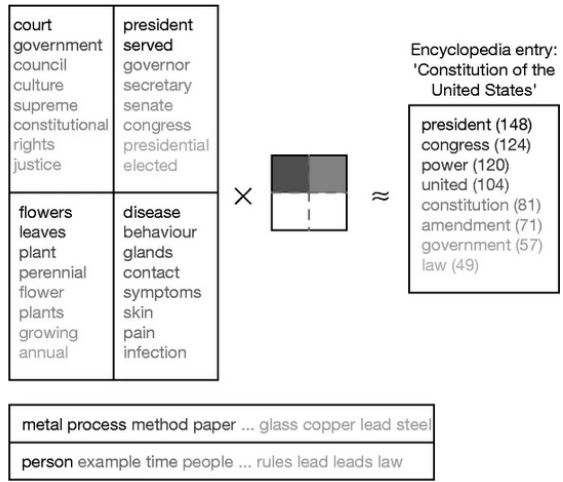
$$W_{ia} \leftarrow W_{ia} \sum_{\mu} \frac{V_{i\mu}}{(WH)_{i\mu}} H_{a\mu}$$

$$W_{ia} \leftarrow \frac{W_{ia}}{\sum_{j} W_{ja}}$$

$$H_{a\mu} \leftarrow H_{a\mu} \sum_{i} W_{ia} \frac{V_{i\mu}}{(WH)_{i\mu}}$$

Figure 4: Non-negative matrix factorization (NMF) discovers semantic features of $m = 30{,}991$ articles from the Grolier encyclopedia.

From: Learning the parts of objects by non-negative matrix factorization



The Objective Function

The **Squared Frobenius Norm**:

$$\min_{\mathbf{W},\mathbf{H}\geq 0} \frac{1}{2}\|\mathbf{X}-\mathbf{W}\mathbf{H}\|_F^2 = \frac{1}{2}\sum_{i,j}(x_{ij}-\sum_k w_{ik}h_{kj})^2$$

Optimization Properties:

- The problem is **not convex** in both $\mathbf{W}$ and $\mathbf{H}$ simultaneously.
- It is **convex** in $\mathbf{W}$ given $\mathbf{H}$ (and vice versa).
- This leads to **Alternating Minimization** strategies.

Optimization Step 1: Alternating Least Squares (ALS)

Since the objective is convex in one matrix when the other is fixed, we can iterate:

1. Fix $\mathbf{H}$, solve $\min_{\mathbf{W} \geq 0} \|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2$
2. Fix $\mathbf{W}$, solve $\min_{\mathbf{H} \geq 0} \|\mathbf{X}^T - \mathbf{H}^T\mathbf{W}^T\|_F^2$

Characteristics:

- Highly parallelizable across columns of $\mathbf{X}$.
- Strict execution needs Non-Negative Least Squares (NNLS) solvers.
- *Fast ALS:* Often implemented by solving standard unconstrained least squares and simply projecting negative values to 0 or ϵ.

Optimization Step 2: Multiplicative Updates

Lee and Seung (1999) proposed an algorithm that ensures non-negativity automatically if initialized with positive values.

Update for $\mathbf{H}$

$$H_{kj} \leftarrow H_{kj} \frac{(\mathbf{W}^T\mathbf{X})_{kj}}{(\mathbf{W}^T\mathbf{W}\mathbf{H})_{kj}}$$

Update for $\mathbf{W}$

$$W_{ik} \leftarrow W_{ik} \frac{(\mathbf{X}\mathbf{H}^T)_{ik}}{(\mathbf{W}\mathbf{H}\mathbf{H}^T)_{ik}}$$

Deriving Multiplicative Updates (Gradient Descent)

Multiplicative updates are a clever rescaling of **Gradient Descent**.

Standard gradient descent for $\mathbf{H}$ involves learning rate η:

$$\mathbf{H} \leftarrow \mathbf{H} - \eta_H \circ \nabla_{\mathbf{H}} J$$

Where the gradient $\nabla_{\mathbf{H}} J = \mathbf{W}^T\mathbf{W}\mathbf{H} - \mathbf{W}^T\mathbf{X}$.

By choosing a *spatially varying* learning rate $\eta_H = \frac{\mathbf{H}}{\mathbf{W}^T\mathbf{W}\mathbf{H}}$, the subtractive term cancels out!

$$\mathbf{H} \leftarrow \mathbf{H} \circ \frac{\mathbf{W}^T\mathbf{X}}{\mathbf{W}^T\mathbf{W}\mathbf{H}}$$

Result: No subtraction occurs. If $\mathbf{H}$ starts positive, it stays positive indefinitely.

Think - Pair - Share: A Full NMF Iteration

Vevox Input Required

Let's perform one **entire alternating update cycle** (update $\mathbf{H}$, then update $\mathbf{W}$). We have a 1D column vector $\mathbf{X}$ and start with terrible guesses: all 1s!

$$\mathbf{X} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}, \quad \mathbf{W}_{old} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{H}_{old} = \begin{bmatrix} 1 \end{bmatrix}$$

Task: 1 full iteration

Step 1: Calculate $\mathbf{H}_{new} = \mathbf{H}_{old} \cdot \frac{\mathbf{W}_{old}^T \mathbf{X}}{\mathbf{W}_{old}^T \mathbf{W}_{old} \mathbf{H}_{old}}$

Step 2: Calculate $\mathbf{W}_{new} = \mathbf{W}_{old} \circ \frac{\mathbf{X}\mathbf{H}_{new}^T}{\mathbf{W}_{old}\mathbf{H}_{new}\mathbf{H}_{new}^T}$ *(Note: Use the $\mathbf{H}_{new}$ you just found!)*

- **Share:** Submit your final vector $\mathbf{W}_{new}$ in Vevox!

Solution: A Full NMF Iteration

Step 1: Update $\mathbf{H}$

Numerator ($\mathbf{W}^T\mathbf{X}$):

$$\begin{bmatrix}1 & 1\end{bmatrix}\begin{bmatrix}4\\6\end{bmatrix} = 10$$

Denominator ($\mathbf{W}^T\mathbf{W}\mathbf{H}_{old}$):

$$\left(\begin{bmatrix}1 & 1\end{bmatrix}\begin{bmatrix}1\\1\end{bmatrix}\right)\cdot[1] = 2$$

$$\mathbf{H}_{new} = 1 \cdot \left(\tfrac{10}{2}\right) = \mathbf{5}$$

Step 2: Update $\mathbf{W}$ *(using $\mathbf{H} = 5$)*

Numerator ($\mathbf{X}\mathbf{H}^T$):

$$\begin{bmatrix}4\\6\end{bmatrix}[5] = \begin{bmatrix}20\\30\end{bmatrix}$$

Denominator ($\mathbf{W}_{old}\mathbf{H}_{new}\mathbf{H}_{new}^T$):

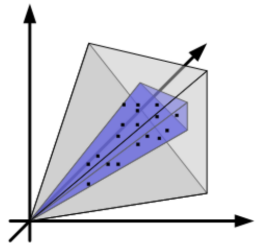
$$\begin{bmatrix}1\\1\end{bmatrix}\cdot(5\cdot 5) = \begin{bmatrix}25\\25\end{bmatrix}$$

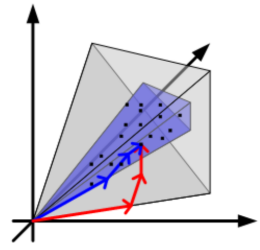
$$\mathbf{W}_{new} = \begin{bmatrix}1\cdot\frac{20}{25}\\1\cdot\frac{30}{25}\end{bmatrix} = \begin{bmatrix}0.8\\1.2\end{bmatrix}$$

Convergence: *We started with terrible guesses (all 1s). After just **one** alternating iteration, let's check the reconstruction:* $\mathbf{W}_{new}\mathbf{H}_{new} = \begin{bmatrix}0.8\\1.2\end{bmatrix}\cdot 5 = \begin{bmatrix}4\\6\end{bmatrix}$.

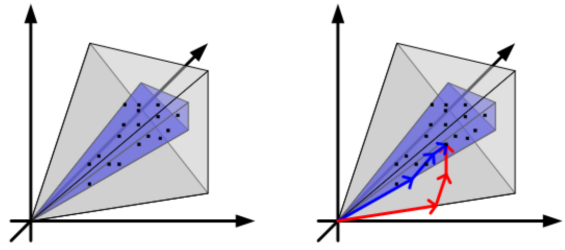
Any concerns with NMF, so far?

Any concerns with I





Any concerns with I



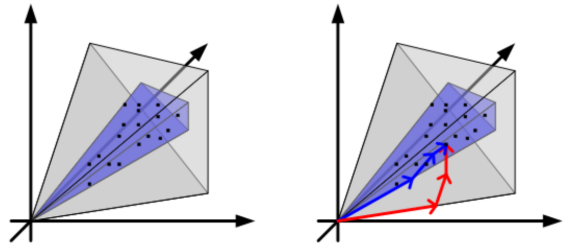
The Ambiguity Problem: Matrix factorization is inherently not unique. For any invertible matrix $\mathbf{Q}$:

$$\mathbf{X} \approx \mathbf{WH} = (\mathbf{WQ}^{-1})(\mathbf{QH}) = \tilde{\mathbf{W}}\tilde{\mathbf{H}}$$

As long as $\tilde{\mathbf{W}} \geq 0$ and $\tilde{\mathbf{H}} \geq 0$, we have found a completely different, equally valid NMF solution!

How can we disambiguate the solution? *(Donoho & Stodden 2003, Laurberg et al. 2008)*

Any concerns with I



The Ambiguity Problem: Matrix factorization is inherently not unique. For any invertible matrix $\mathbf{Q}$:

$$\mathbf{X} \approx \mathbf{WH} = (\mathbf{WQ}^{-1})(\mathbf{QH}) = \tilde{\mathbf{W}}\tilde{\mathbf{H}}$$

As long as $\tilde{\mathbf{W}} \geq 0$ and $\tilde{\mathbf{H}} \geq 0$, we have found a completely different, equally valid NMF solution!

How can we disambiguate the solution? *(Donoho & Stodden 2003, Laurberg et al. 2008)*

- **Geometric Constraints:** Algorithms that minimize the geometric *volume* spanned by the columns of $\mathbf{W}$ force a unique, tightly fitting positive cone around the data.
- **Sparsity Constraints:** Enforcing strict L_1 penalties forces 'structural zeros' in the matrices. This drastically shrinks the mathematical space of valid $\mathbf{Q}$ transformations.

Resource demo

https://www.audiolabs-erlangen.de/resources/MIR/FMP/C8/C8S3_NMFbasic.html

- Non-negative Matrix Factorization
- Independent Component analysis
- Archetypal Analysis
- Sparse coding

Independent Component Analysis

Blind Source Separation: Assume we observe a set of signals $\mathbf{x}(t)$ which are linear mixtures of unknown independent sources $\mathbf{s}(t)$:

$$\mathbf{x} = \mathbf{A}\mathbf{s}$$

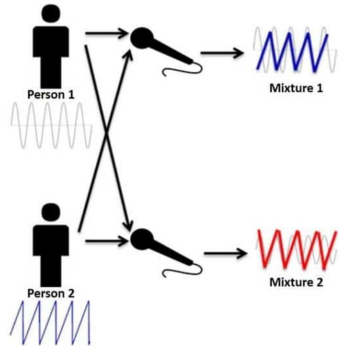


Figure: source: https://vocal.com/blind-signal-separation/independent-component-analysis/

The Cocktail Party Application

Imagine two microphones recording two different people speaking.

- Each microphone receives a blend: $x_1 = a_{11}s_1 + a_{12}s_2$ and $x_2 = a_{21}s_1 + a_{22}s_2$.
- **PCA** would fail because it only looks for orthogonal directions of variance; it wouldn't separate the speakers.
- **ICA** uses the "non-Gaussian" nature of human speech (high kurtosis) to untangle the voices.

ICA effectively "rotates" the space until the projections look as non-Gaussian as possible.

Independent Component Analysis

The Goal: Find the un-mixing matrix $\mathbf{W} \approx \mathbf{A}^{-1}$ such that we recover the sources:

$$\hat{\mathbf{s}} = \mathbf{W}\mathbf{x}$$

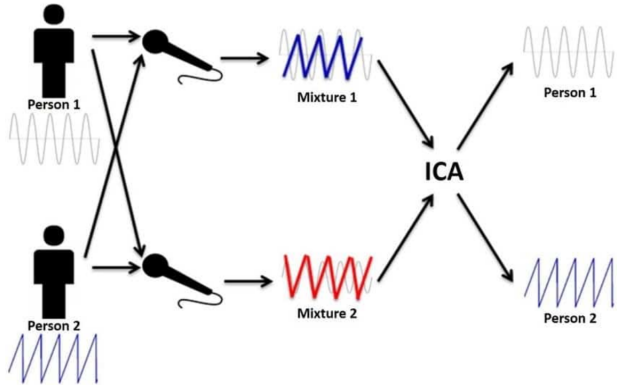


Figure: source: https://vocal.com/blind-signal-separation/independent-component-analysis/

Assumptions and Non-Gaussianity

The Key Assumptions:

1. The components s_i are **statistically independent**.
2. The components s_i have **non-Gaussian distributions**.

Assumptions and Non-Gaussianity

The Key Assumptions:

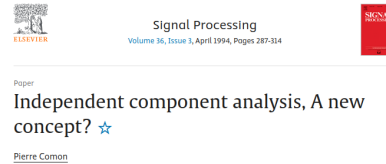
1. The components s_i are **statistically independent**.
2. The components s_i have **non-Gaussian distributions**.

ICA relies on a fundamental statistical concept:
The Central Limit Theorem (CLT).

- The sum of independent variables tends toward a Gaussian distribution.
- Therefore, a mixture $x = \sum a_i s_i$ is 'more Gaussian' than the original sources s_i.

The Strategy:

To recover the original sources, we find the matrix $\mathbf{W}$ that **maximizes the non-Gaussianity** of the estimated signals.



Measuring Non-Gaussianity

How do we mathematically quantify 'how non-Gaussian' a signal is?

Measuring Non-Gaussianity

How do we mathematically quantify 'how non-Gaussian' a signal is?

- **Excess Kurtosis:** The 4th standardized moment minus 3.

$$\text{Excess Kurtosis} = \frac{\mu^4}{\sigma^4} - 3$$

- D: Laplace distribution, a.k.a. double exponential distribution, red curve, excess kurtosis = 3
- S: hyperbolic secant distribution, orange curve, excess kurtosis = 2
- L: logistic distribution, green curve, excess kurtosis = 1.2

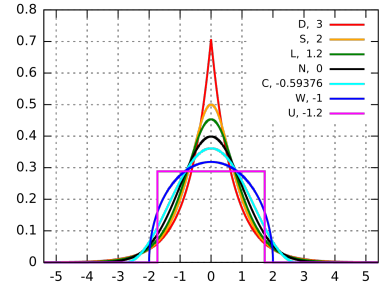


Figure: Source: Wikipedia

- N: normal distribution, black curve, excess kurtosis = 0
- C: raised cosine distribution, cyan curve, excess kurtosis = -0.593762...
- W: Wigner semicircle distribution, blue curve, excess kurtosis = -1
- U: uniform distribution, magenta, excess kurtosis = -1.2.

Optimization Step 1: Pre-processing (Whitening)

Before optimizing for non-Gaussianity, we drastically simplify the problem by **centering** and **whitening** the observed data $\mathbf{x}$.

- **Centering:** Subtract the mean to make $\mathbf{x}$ a zero-mean variable.
- **Whitening (Sphering):** Apply a linear transformation (often via PCA) so that the components of the new vector $\tilde{\mathbf{x}}$ are uncorrelated and have unit variance:

$$E[\tilde{\mathbf{x}}\tilde{\mathbf{x}}^T] = \mathbf{I}$$

Why Whiten? It reduces the search for the unmixing matrix $\mathbf{W}$ to finding an **orthogonal matrix**. Instead of searching all possible parameter matrices, we only need to find the correct *rotations*. Thus ICA mounts to solving for an orthonormal matrix A such that $S = A^T Y$ (are independent (and non-Gaussian).)

Optimization Step 2: The FastICA Algorithm

Proposed by Hyvärinen (1999), **FastICA** efficiently maximizes non-Gaussianity. Instead of taking slow, fixed steps (gradient ascent), it uses a fast Newton-like method.

Conceptually:

1. Step in the direction of maximum non-Gaussianity.
2. 'Snap' the vector back to the surface of our whitened sphere.

FastICA Iteration Rule (for one row of $\mathbf{W}$)

$$\mathbf{w}_{new} \leftarrow \underbrace{E[\tilde{\mathbf{x}} g(\mathbf{w}^T \tilde{\mathbf{x}})]}_{\text{Search Direction}} - \underbrace{E[g'(\mathbf{w}^T \tilde{\mathbf{x}})]\mathbf{w}}_{\text{Newton Step Correction}}$$

- $g(\cdot)$ measures non-Gaussianity (e.g., $g(u) = \tanh(a_1 u)$ handles outliers well).
- **Crucial Step - Normalize:** $\mathbf{w} \leftarrow \mathbf{w}/\|\mathbf{w}\|$. This ensures we stay on the whitened sphere, meaning we are strictly finding *rotations*.
- **Speed:** Converges vastly faster (cubic/quadratic) than ordinary gradient descent.

Extracting Multiple Independent Components

How do we find *all* n independent sources without accidentally finding the same signal twice?

Because our data is whitened, the unmixing vectors $\mathbf{w}_1, \ldots, \mathbf{w}_n$ **must be strictly perpendicular (orthogonal)** to each other.

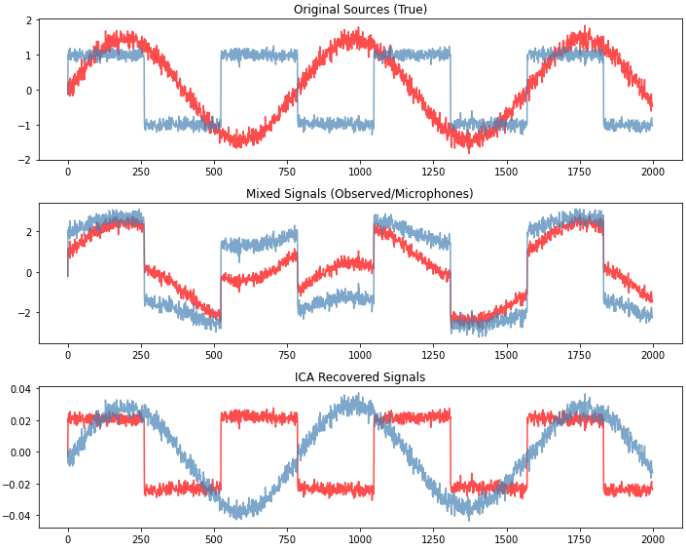
Two Main Strategies:

- **1. Deflationary Approach (One-by-one):**
  - Find the first component $\mathbf{w}_1$.
  - When searching for $\mathbf{w}_2$, mathematically *erase* (project out) any part of $\mathbf{w}_1$ from the search space before normalizing:

$$\mathbf{w}_2 \leftarrow \mathbf{w}_2 - (\mathbf{w}_2^T \mathbf{w}_1)\mathbf{w}_1$$

- **2. Symmetric Approach (All at once):**
  - Optimize all $\mathbf{w}_i$ vectors simultaneously.
  - At each step, symmetrically push all vectors apart so they remain perfectly orthogonal to one another.

Example


Original Sources (True)
Mixed Signals (Observed/Microphones)
ICA Recovered Signals

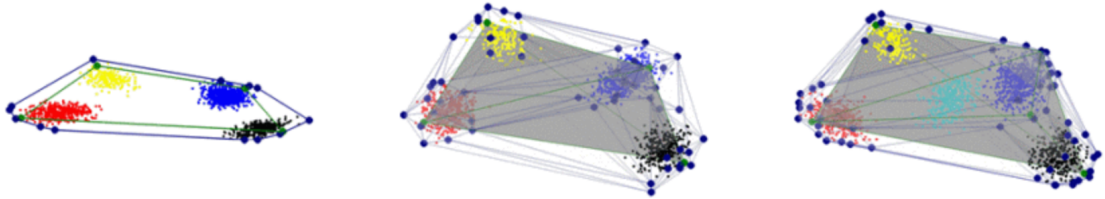
- Non-negative Matrix Factorization
- Independent Component analysis
- Archetypal Analysis
- Sparse coding

Archetypal Analysis (AA): The Intuition

While methods like PCA or K-means focus on the **average** or the **centroid** of the data, Archetypal Analysis focuses on the extremes.

The Concept: Instead of finding a 'Typical' representative, we find the 'prototypes' that define the boundaries of the dataset.

- Every observation is described as a **convex mixture** of these extreme 'Archetypes'.



Convex hull: Blue lines and light shaded region
Dominant convex hull: green lines and gray shaded region

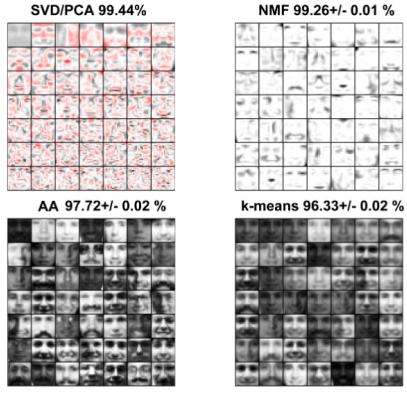


Fig. 3. Properties of features extracted based on SVD/PCA, NMF, AA and K-means. Whereas SVD/PCA extract low to high spatial frequency components and NMF decompose the data into an atomic mixture of constituting parts the AA model extracts notably more distinct aspects than K-means.

Mørup, M., & Hansen, L. K. (2010, August). Archetypal analysis for machine learning. In 2010 IEEE International Workshop on Machine Learning for Signal Processing (pp. 172-177). IEEE.

Audio Application: Characterizing Speech Consider an acoustic feature space (e.g., Pitch vs. Spectral Flux).

- **PCA:** Finds the 'average' vocal profile and directions of variation.
- **NMF:** Finds the 'spectral parts' (harmonics, noise).
- **AA:** Finds the **pure states** of the database.

Example Archetypes:

- **A1:** The purest 'Deep/Bass' voice.
- **A2:** The purest 'High-pitched' Shriek.
- **A3:** The purest 'Whisper/Noise' state.

Every other recording is quantified by its proximity to these three corners.

Archetypal Analysis: The Objective Function

To mathematically find these 'extremes' (archetypes) and use them for reconstruction, AA minimizes the reconstruction error (where $\mathbf{X} = \mathbf{XSH} + \mathbf{E}$):

The AA Loss Function

$$\min_{\mathbf{S},\mathbf{H}} \|\mathbf{E}\|_F^2 = \min_{\mathbf{S},\mathbf{H}} \|\mathbf{X} - \mathbf{XSH}\|_F^2$$

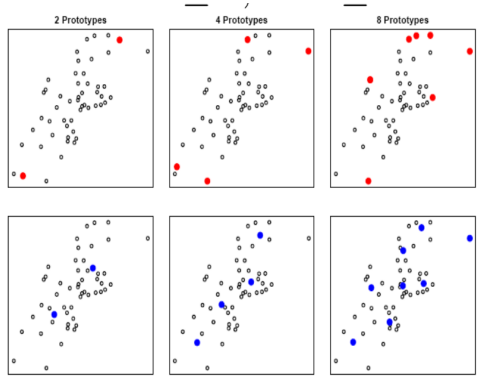
The Two Core Constraints:

- **$\mathbf{S}$ Matrix (Archetype Formation):** $s_{ij} \geq 0$ and $\sum_i |s_{ij}| = 1$.
  Forces archetypes to be strictly composed as convex combinations (percentages) of real, existing data points.

- **$\mathbf{H}$ Matrix (Data Reconstruction):** $h_{ij} \geq 0$ and $\sum_i |h_{ij}| = 1$.
  Forces the reconstructed data to be strictly composed as convex combinations (percentages) of the newly found archetypes.

Prototype

$$\mathbf{w}_d = \mathbf{X}\mathbf{s}_d$$



Blue dots: k-means prototypes, red dots: Archetypal Analysis prototypes

Deconstructing the Math: Why $\mathbf{XSH}$?

Why not use standard matrix factorization ($\mathbf{X} \approx \mathbf{WH}$)?

AA enforces a strict **two-step** geometric leash to ensure archetypes aren't just imaginary points floating in space:

1. **Step 1: Define the Archetypes ($\mathbf{Z}$)**

$$\mathbf{Z} = \mathbf{XS}$$

Archetypes ($\mathbf{Z}$) cannot be arbitrary. They are built directly by blending actual data features ($\mathbf{X}$) using coefficient weights ($\mathbf{S}$).

2. **Step 2: Reconstruct the Data ($\hat{\mathbf{X}}$)**

$$\hat{\mathbf{X}} = \mathbf{ZH}$$

Every original data point is then approximated as a fractional mixture ($\mathbf{H}$) of our newly defined extreme archetypes ($\mathbf{Z}$).

The Result: Substituting $\mathbf{Z}$ gives us our final approximation: $\hat{\mathbf{X}} = (\mathbf{XS})\mathbf{H} = \mathbf{XSH}$. The residual error is captured by the matrix $\mathbf{E}$.

- Non-negative Matrix Factorization
- Independent Component analysis
- Archetypal Analysis
- Sparse coding

What is Sparse Coding?

Sparse coding is a method for finding an **overcomplete** basis (dictionary) to represent data using as few active components as possible.

The Model: $\mathbf{x} \approx \mathbf{W}\mathbf{h}$

- $\mathbf{W} \in \mathbb{R}^{I \times K}$: The **Dictionary**.
- $\mathbf{h} \in \mathbb{R}^{K}$: The **Sparse Code**.
- **Overcomplete:** $K > I$ (More basis vectors than dimensions).

The Goal: Reconstruct $\mathbf{x}$ accurately while keeping most elements of $\mathbf{h}$ equal to zero.

The Bio-Hypothesis (Olshausen & Field, 1996)

Sparse coding was originally proposed as a model for how the brain's **Primary Visual Cortex** processes information.

- The brain has millions of neurons, but only a tiny fraction fire in response to a specific image.
- **Efficiency:** Sparse representations are energy-efficient and make high-level features (like edges) easier to detect.
- When applied to natural images, Sparse Coding automatically learns 'Gabor-like' edge detectors, matching actual receptive fields in the brain.

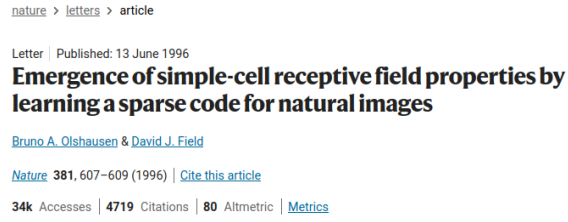


Figure: *Nature* (1996)

The Objective Function

We minimize a loss function consisting of a **reconstruction error** and a **sparsity penalty**:

$$\mathcal{L}(\mathbf{W}, \mathbf{H}) = \underbrace{\frac{1}{2}\|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2}_{\text{Reconstruction}} + \underbrace{\lambda \sum_j \|\mathbf{h}_j\|_1}_{\text{Sparsity Penalty}}$$

- The L_1 norm ($\sum |h_i|$) acts as a convex proxy for the L_0 norm (counting non-zeros).
- **Shrinkage:** The L_1 penalty 'pushes' small coefficients to exactly zero.

Optimization Strategy: Alternating Minimization

The objective is **not joint-convex** in $\mathbf{W}$ and $\mathbf{H}$, but it is convex in one if the other is fixed. We alternate between two steps:

Step 1: Sparse Coding (Fix $\mathbf{W}$, update $\mathbf{H}$)

The problem decouples into independent optimization problems for each data vector $\mathbf{x}_j$:

$$\min_{\mathbf{h}_j} \frac{1}{2}\|\mathbf{x}_j - \mathbf{W}\mathbf{h}_j\|_2^2 + \lambda\|\mathbf{h}_j\|_1$$

*This is exactly the **Lasso** problem!*

Optimization Strategy: Alternating Minimization

The objective is **not joint-convex** in $\mathbf{W}$ and $\mathbf{H}$, but it is convex in one if the other is fixed. We alternate between two steps:

Step 1: Sparse Coding (Fix $\mathbf{W}$, update $\mathbf{H}$)

The problem decouples into independent optimization problems for each data vector $\mathbf{x}_j$:

$$\min_{\mathbf{h}_j} \frac{1}{2}\|\mathbf{x}_j - \mathbf{W}\mathbf{h}_j\|_2^2 + \lambda\|\mathbf{h}_j\|_1$$

*This is exactly the **Lasso** problem!*

Step 2: Dictionary Update (Fix $\mathbf{H}$, update $\mathbf{W}$)

$$\min_{\mathbf{W}} \frac{1}{2}\|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2 \quad \text{subject to } \|\mathbf{w}_k\|_2^2 \leq 1$$

Why the constraint? Without it, the model could infinitely scale up $\mathbf{W}$ and scale down $\mathbf{H}$ to drive the L_1 penalty to zero without changing the reconstruction.

Solving Step 1: Standard Algorithms

Because Step 1 is mathematically identical to the **Lasso** problem, standard gradient descent fails due to the non-differentiable L_1 penalty at zero. We leverage specialized algorithms:

1. Coordinate Descent (The Modern Standard)

Optimizes one coordinate of $\mathbf{H}$ at a time. (lecture 3)

2. LARS (Least Angle Regression)

A stage-wise algorithm that adds active variables one by one. (lecture 3)

Choose the number of components

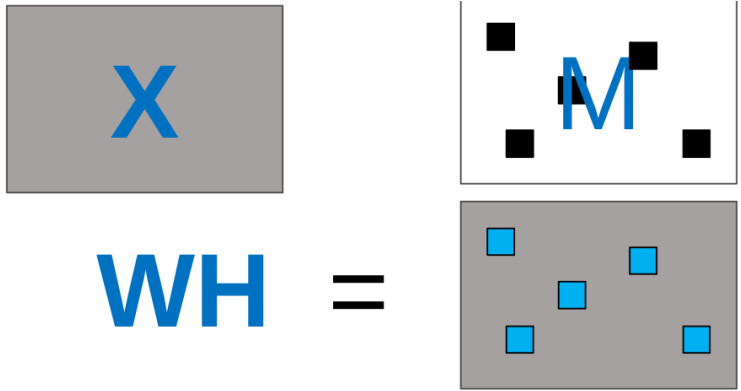
Cross-Validation in Unsupervised Decompositions:
In supervised learning, we hold out entire rows (samples). In unsupervised models (AA, NMF), removing a row means the model cannot learn the specific mixture weights (e.g., the $\mathbf{H}$ matrix) needed to reconstruct it later.

The Solution: 'Speckled' CV (Matrix Masking)

Instead of removing whole rows or columns, we hold out individual *elements* randomly scattered throughout the matrix $\mathbf{X}$.

1. **Mask:** Randomly hide a percentage of entries in $\mathbf{X}$ (treat as missing).
2. **Fit:** Train the algorithm for k components, explicitly ignoring the masked data during the loss calculation.
3. **Reconstruct:** Multiply the resulting matrices (e.g: $\hat{\mathbf{X}} = \mathbf{XSH}$) to generate predictions for the entire matrix, including missing spots.
4. **Evaluate:** Calculate the MSE *only* on the originally masked entries:

$$\text{CV Error} = \sum_{\text{masked } i,j} (\mathbf{X}_{ij} - \hat{\mathbf{X}}_{ij})^2$$



Least squares objective function:

$$\frac{1}{2}\|(\mathbf{1}-\mathbf{M})*(\mathbf{X}-\mathbf{WH})\|_F^2 = \frac{1}{2}\sum_{i,j}(1-m_{i,j})(x_{i,j}-(\mathbf{WH})_{i,j})^2$$

Prediction error

$$\frac{1}{2}\|(\mathbf{M}*(\mathbf{X}-\mathbf{WH})\|_F^2 = \frac{1}{2}\sum_{i,j} m_{i,j}(x_{i,j}-(\mathbf{WH})_{i,j})^2$$

Modern Bridge: Mechanistic Interpretability

The Problem: Polysemanticity
LLM 'neurons' fire for multiple, unrelated concepts (e.g: Canada and Syntax Errors). This makes the models unreadable black boxes.

The Solution: Sparse Autoencoders
By training a sparse coder on the internal activations of an LLM, researchers can untangle these mixed signals into a readable dictionary of **monosemantic** (single-concept) features.

`https://transformer-circuits.pub/2023/monosemantic-features/index.html`